

Sistema de Facturación para Farmacia

Rodríguez Santana Andros · Carlos Soto Solórzano · Alejandro Salas Sánchez

Universidad Fidélitas — San Pedro, Costa Rica

arodriguez50764@ufide.ac.cr · csoto60826@ufide.ac.cr · Asalas20395@ufide.ac.cr

Resumen—Este proyecto presenta el desarrollo e implementación de un sistema web de facturación para una farmacia. El sistema integra la gestión de usuarios y roles, clientes, medicamentos, proveedores, inventario, carrito de compras, facturación y reportes. Fue desarrollado con Java y Spring Boot, utilizando Spring Security para el control de acceso, Spring Data JPA para la persistencia de datos, Thymeleaf y Bootstrap 5 para las vistas y MySQL como base de datos. La aplicación diferencia las funciones disponibles para administrador, empleado y cliente, mantiene actualizado el stock después de cada venta y permite consultar información sobre facturación y medicamentos vendidos.

Abstract—This project presents the development and implementation of a web-based billing system for a pharmacy. The system integrates user and role management, customers, medications, suppliers, inventory, a shopping cart, billing, and reports. It was developed using Java and Spring Boot, with Spring Security for access control, Spring Data JPA for data persistence, Thymeleaf and Bootstrap 5 for the views, and MySQL as the database. The application provides different functions for administrators, employees, and customers, keeps inventory stock updated after each sale, and allows users to consult billing information and sold medications.

Palabras clave—Spring Boot, farmacia, facturación, inventario, seguridad, MySQL, roles.

Prueba	Resultado
CRUD de medicamentos	Create, Read, Update y Delete verificados.
Compra y factura	Factura generada y disponible para consulta.
Inventario	Stock actualizado después de la venta.
Roles	Rutas no autorizadas muestran acceso denegado.
Historial	Cliente visualiza sus compras.
Reportes	Ventas diarias y medicamentos más vendidos disponibles.

I. INTRODUCCIÓN

Las farmacias manejan información que cambia constantemente: clientes, medicamentos, existencias, proveedores y ventas. Cuando estos datos se llevan en procesos separados, resulta más difícil conocer el stock real, consultar una factura anterior o controlar quién puede modificar información administrativa. A partir de esta necesidad se desarrolló Farmacia Vida, una aplicación web orientada a centralizar las tareas principales de una farmacia en un mismo sistema.

El proyecto no se limita a almacenar registros. También incorpora un flujo completo de compra en el que el usuario consulta el catálogo, agrega medicamentos al carrito, confirma los datos de la compra y obtiene una factura. Al finalizar la venta, el inventario se actualiza para mantener correspondencia entre las unidades vendidas y las existencias disponibles. De esta forma, los módulos de tienda, facturación e inventario trabajan sobre la misma información.

Otro aspecto importante fue separar las funciones según el tipo de usuario. El sistema trabaja con los roles ADMIN, EMPLEADO y CLIENTE. El administrador dispone de las opciones de gestión y configuración; el empleado utiliza las funciones operativas relacionadas con clientes, inventario y ventas; y el cliente accede a la tienda, al carrito y a su propio historial de compras. Esta separación permite demostrar autenticación y autorización dentro de un caso de uso real.

El objetivo general fue implementar una aplicación web funcional para apoyar la facturación y el control administrativo de una farmacia, utilizando Spring Boot y una arquitectura organizada por capas. El alcance incluye autenticación, control de acceso por roles, gestión de medicamentos, clientes y proveedores, inventario, carrito, facturación, reportes, persistencia en MySQL y despliegue en producción.

II. MARCO TEÓRICO / REVISIÓN DE LITERATURA

A. Spring Boot y arquitectura por capas

Spring Boot se utilizó como base de la aplicación porque permite organizar un proyecto Java web mediante componentes con responsabilidades definidas. En Farmacia Vida, las solicitudes pasan desde las vistas hacia los controladores; posteriormente interviene la capa de servicios, que concentra la lógica de negocio, y finalmente los repositorios realizan las operaciones de persistencia. Esta separación facilita ubicar cada responsabilidad y evita concentrar toda la lógica en una sola clase.

El flujo general implementado puede resumirse como Vista Thymeleaf → Controller → Service → Repository → MySQL. Los controladores atienden las rutas HTTP, los servicios coordinan operaciones como el registro de una venta y la actualización del stock, y los repositorios trabajan con las entidades persistentes mediante Spring Data JPA.

B. Persistencia con Spring Data JPA y MySQL

Spring Data JPA se empleó para relacionar las clases del proyecto con las tablas de la base de datos. El modelo incluye usuarios, proveedores, medicamentos, clientes, elementos del carrito, ventas y detalles de venta. Las relaciones entre estas entidades permiten conservar el detalle histórico de cada factura y, al mismo tiempo, mantener la información del inventario asociada a los medicamentos.

MySQL funciona como motor de base de datos. En producción, la base de datos se encuentra alojada de forma remota, por lo que la aplicación desplegada puede conservar la información sin depender del equipo local utilizado durante el desarrollo.

C. Seguridad y vistas dinámicas

Spring Security se integró para controlar el inicio de sesión y la autorización. La seguridad no se aplicó únicamente ocultando enlaces en la interfaz: las rutas administrativas también se encuentran restringidas. Durante las pruebas se verificó que un cliente o un empleado que intenta acceder directamente a una función reservada recibe la pantalla de acceso denegado.

Las vistas fueron construidas con Thymeleaf, HTML, CSS, JavaScript y Bootstrap 5. Thymeleaf permite mostrar información proveniente del servidor y adaptar elementos de navegación según el usuario autenticado. La interfaz mantiene

una estructura común de menú lateral y módulos diferenciados para administración, operación y compra.

III. METODOLOGÍA

El desarrollo se realizó de manera incremental a partir de veinte historias de usuario definidas para el proyecto. Estas historias abarcan registro y consulta de clientes, gestión de medicamentos y proveedores, inventario, alertas de stock, facturación, usuarios, roles, reportes e historial de ventas. A partir de ellas se priorizaron primero las entidades y operaciones CRUD, y posteriormente se integraron los flujos que dependen de varios módulos.

La primera etapa consistió en estructurar el proyecto Spring Boot y configurar la conexión con MySQL. Después se desarrollaron las entidades, repositorios, servicios y controladores necesarios para los módulos administrativos. Una vez establecida esa base se incorporaron las vistas con Thymeleaf y se ajustó la navegación para que cada módulo pudiera probarse desde la aplicación.

La siguiente etapa se enfocó en seguridad. Se definieron los perfiles ADMIN, EMPLEADO y CLIENTE y se restringieron las funciones según las responsabilidades de cada uno. Posteriormente se implementó el flujo de tienda y carrito, seguido por la generación de facturas y la actualización automática de existencias. Finalmente se agregaron reportes y se preparó la configuración para ejecutar la aplicación fuera del entorno local.

Las pruebas se realizaron sobre los flujos completos y no solamente sobre pantallas aisladas. Se verificó el registro e inicio de sesión, el acceso por rol, el CRUD de medicamentos y proveedores, la consulta de inventario, la compra desde el carrito, la generación de facturas, el descuento de stock y la consulta de reportes. También se probaron accesos directos a rutas restringidas para comprobar que la autorización se aplicara en el servidor.

Como control de versiones se utilizó Git y GitHub. Las evidencias finales se conservaron en la carpeta evidencias del repositorio para documentar las pruebas realizadas. La versión de producción se publicó en Render y utiliza un servicio MySQL alojado en Aiven.

A. Organización del trabajo y validación de requerimientos

Las historias de usuario sirvieron como guía para comprobar que cada módulo tuviera una finalidad concreta. Por ejemplo, las historias relacionadas con medicamentos no se consideraron completas únicamente por mostrar una lista: se verificó que existieran las operaciones de creación, consulta, edición y eliminación, además de las validaciones de precio y stock. En facturación se comprobó el proceso completo desde la selección de productos hasta la generación del documento y la actualización de existencias.

La validación se realizó de forma continua durante el desarrollo. Después de incorporar un módulo se probaba su navegación, persistencia y comportamiento con diferentes perfiles. Este procedimiento permitió detectar problemas de permisos, redirecciones y visualización antes del despliegue final. Las correcciones se registraron mediante commits para conservar un historial de los cambios realizados.

B. Criterios de prueba

Para las pruebas funcionales se utilizaron casos positivos y negativos. Entre los casos positivos se incluyeron el inicio de sesión con credenciales válidas, el registro de información, la edición de medicamentos, la creación de una compra y la consulta de una factura. Entre los casos negativos se probaron accesos a rutas sin el rol necesario, cantidades superiores a las permitidas y formularios con información incompleta.

También se revisó la consistencia de los datos después de cada operación. Una venta no se consideró correcta solamente porque apareciera la factura en pantalla; se verificó además que el registro quedara disponible en el historial, que el detalle mantuviera las cantidades vendidas y que el inventario mostrara el nuevo stock. Esta comprobación fue especialmente importante porque varios módulos dependen de la misma transacción.

IV. DESARROLLO

A. Modelo de datos

El modelo relacional se diseñó alrededor del proceso de venta. Un proveedor puede estar relacionado con varios medicamentos; los medicamentos pueden formar parte de elementos de carrito y detalles de venta; un usuario puede registrar ventas y mantener elementos de carrito; y cada venta conserva uno o varios detalles. La entidad Cliente se utiliza para la información comercial del comprador, mientras que Usuario representa la cuenta utilizada para autenticarse en el sistema.

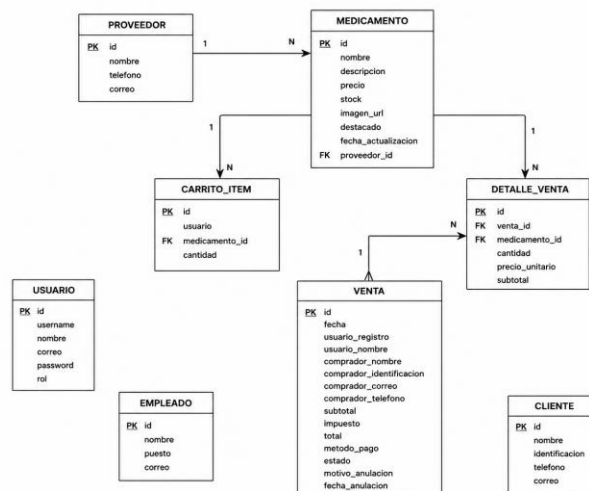


Fig. 1. Diagrama entidad-relación final de Farmacia Vida.

B. Gestión administrativa

El administrador dispone de un dashboard con un resumen de medicamentos en catálogo, productos con stock bajo, clientes registrados y proveedores activos. Desde el menú puede ingresar a usuarios, medicamentos, clientes, proveedores, inventario, facturas y reportes. En usuarios es posible consultar las cuentas registradas y modificar el rol asignado.

El módulo de medicamentos implementa las operaciones Create, Read, Update y Delete. Cada registro muestra nombre, precio, stock y proveedor, y se identifican visualmente los productos con pocas existencias. El módulo de proveedores utiliza el mismo enfoque de mantenimiento y permite registrar y actualizar los datos de contacto de los distribuidores.

C. Compra, facturación e inventario

Para el cliente, el proceso comienza en la tienda. Los medicamentos disponibles se muestran con su precio y opción para agregar una cantidad al carrito. En el carrito se puede modificar la cantidad, eliminar productos, vaciar la compra o continuar al pago. Antes de confirmar se solicitan los datos del comprador y el método de pago, y se presenta un resumen con los productos y el total.

Cuando se confirma la compra, el sistema crea la venta y sus detalles, genera un número de factura y calcula subtotal, impuesto y total. La factura muestra el comprador, la fecha, el usuario que registró la operación, el método de pago y cada medicamento vendido. Al mismo tiempo, las cantidades se descuentan del stock. Esta integración evita que la factura y el inventario queden como procesos independientes.

El historial de facturas cambia según el perfil. El personal autorizado puede consultar las ventas correspondientes a su función, mientras que el cliente visualiza sus propias compras. La factura puede abrirse nuevamente para consultar el detalle y dispone de una opción de impresión.

D. Reportes

El módulo de reportes permite consultar las ventas de un día específico y muestra tanto la cantidad de operaciones como el monto total vendido. También incluye una consulta de medicamentos más vendidos dentro de un rango de fechas. Con ello se aprovecha la información almacenada en las ventas para obtener resultados útiles para la administración.

E. Despliegue

La aplicación se encuentra publicada en un servicio web de Render y utiliza una base de datos MySQL en Aiven. Las credenciales y valores propios del entorno de producción se manejan mediante configuración externa al código fuente. Esto permite mantener el mismo proyecto en GitHub y cambiar la conexión según el ambiente de ejecución.

F. Controladores, servicios y repositorios

Los controladores funcionan como punto de entrada de las acciones realizadas desde las vistas. Cada ruta dirige la solicitud al módulo correspondiente y prepara la información que Thymeleaf necesita mostrar. La lógica que implica reglas del negocio se mantiene fuera de las vistas, evitando realizar cálculos o cambios de inventario directamente desde el HTML.

Los servicios coordinan operaciones que requieren más de una acción. En una compra, por ejemplo, se deben validar los productos, registrar la venta, crear sus detalles y modificar las existencias. Los repositorios se encargan de la comunicación con la base de datos a través de Spring Data JPA. Esta distribución hace que el flujo del sistema sea más fácil de seguir durante una defensa técnica, porque cada capa tiene una responsabilidad reconocible.

G. API REST y pruebas con Postman

Como parte de la demostración técnica se contempla la consulta de la API REST mediante Postman. La API permite comprobar el intercambio de información entre el servidor y un cliente distinto de las vistas web. Las solicitudes se prueban observando la ruta utilizada, el método HTTP, los datos enviados cuando corresponda y la respuesta obtenida por el servidor.

Esta prueba complementa la demostración de la interfaz, ya que permite evidenciar que el proyecto no depende únicamente de formularios Thymeleaf. Durante la presentación se puede mostrar una solicitud válida y explicar cómo el controlador recibe la petición, utiliza la capa correspondiente y devuelve una respuesta. De esta manera se relaciona la demostración de Postman con la arquitectura implementada en el proyecto.

H. Validaciones y manejo de errores

Los formularios del sistema requieren información coherente antes de guardar datos. En los módulos administrativos se validan campos obligatorios y valores relacionados con precio o existencias. En el proceso de compra también se controla que la cantidad solicitada pueda ser atendida con el stock disponible. Estas validaciones reducen la posibilidad de registrar información incompleta o producir inconsistencias entre la venta y el inventario.

El manejo de acceso no autorizado se presenta mediante una vista propia de acceso denegado. Esto mejora la respuesta al usuario frente a una ruta restringida y, al mismo tiempo, permite demostrar que la aplicación reconoce la diferencia entre estar autenticado y tener permiso para ejecutar una acción. En producción también se evita exponer información técnica innecesaria al usuario final.

I. Interfaz y navegación por perfil

La interfaz utiliza un menú lateral que cambia de acuerdo con el perfil autenticado. En el administrador aparecen las opciones de usuarios, medicamentos, clientes, proveedores, inventario y reportes. El empleado conserva las funciones necesarias para su trabajo operativo, mientras que el cliente encuentra principalmente tienda, carrito y sus compras. La intención fue que la navegación reflejara las mismas reglas aplicadas por la seguridad del servidor.

El dashboard administrativo resume información relevante sin obligar a recorrer cada módulo. Se muestran cantidades de medicamentos, alertas de stock bajo, clientes registrados y proveedores activos. En las tablas se incorporaron filtros de búsqueda y acciones directas para facilitar la consulta durante el uso y durante la demostración del proyecto.

J. Módulo de usuarios y roles

El módulo de usuarios se reservó para el administrador porque modifica directamente la forma en que una cuenta puede utilizar el sistema. La pantalla presenta el nombre, usuario, correo y rol actual, además de una opción para cambiar el perfil. Esta función se relaciona con la seguridad de la aplicación: modificar un rol no es solamente un cambio visual, sino que determina las rutas y acciones que el usuario podrá ejecutar después de autenticarse.

El registro público crea cuentas destinadas al uso como cliente. Con ello se evita que una persona pueda asignarse por sí misma permisos administrativos. Los perfiles con mayores privilegios se administran desde el área protegida. Esta decisión mantiene separado el proceso normal de creación de una cuenta del mantenimiento de permisos realizado por el administrador.

K. Clientes y relación con la facturación

El módulo de clientes concentra la información necesaria para identificar a los compradores dentro de las operaciones administrativas. En el proceso de venta se manejan datos

como nombre, identificación, correo y teléfono, que posteriormente aparecen en la factura. Esto permite que el documento generado conserve información suficiente para reconocer a quién corresponde la compra.

Durante el desarrollo se procuró que la información mostrada en la factura proviniera del proceso de compra y no de valores escritos directamente en la plantilla. Así, diferentes ventas pueden conservar compradores distintos y el historial puede presentar cada operación con sus propios datos. Esta característica fue comprobada mediante facturas generadas tanto por un empleado como por una cuenta cliente.

L. Inventario y alertas de stock

El inventario presenta una vista de consulta enfocada en las existencias actuales. Además del nombre y precio del medicamento, muestra el proveedor y el estado de disponibilidad. Cuando la cantidad llega a un nivel bajo, el sistema utiliza una alerta visual para facilitar su identificación. El dashboard administrativo resume también cuántos productos se encuentran en esta condición.

La actualización del stock se relaciona directamente con la facturación. Antes de completar una compra se utiliza la cantidad solicitada y, después de registrar la venta, esa cantidad deja de estar disponible. Esta regla evita que el inventario sea únicamente una tabla informativa y lo convierte en parte del flujo de negocio. En las pruebas finales se comparó el valor anterior y posterior a una compra para comprobar el descuento.

M. Factura e historial

La factura se diseñó como una representación legible de la venta. Incluye un número consecutivo visible, estado, fecha de emisión, persona que registró la operación, datos del comprador, método de pago y una tabla con descripción, cantidad, precio unitario y subtotal. Al final se presentan subtotal, IVA y total. También existe una acción de impresión para facilitar su presentación durante la demostración.

El historial evita que la factura exista solamente al finalizar el checkout. Una vez creada, puede consultarse nuevamente desde el módulo correspondiente. El administrador y el personal autorizado pueden revisar las operaciones necesarias para su trabajo, mientras que el cliente obtiene una vista limitada a sus propias compras. Esta diferencia fue una de las pruebas utilizadas para validar simultáneamente persistencia y autorización.

N. Reportes como consulta de la información registrada

El reporte de ventas por día permite seleccionar una fecha y obtener las facturas asociadas, la cantidad de ventas realizadas y el total vendido. Cada fila conserva una opción para abrir el detalle. Esta consulta facilita relacionar el resumen numérico con las facturas que lo originan y permite verificar manualmente el resultado durante una demostración.

El reporte de medicamentos más vendidos utiliza un rango de fechas y presenta los productos ordenados por cantidad. Su finalidad dentro del proyecto es demostrar que los detalles almacenados en las ventas pueden utilizarse para generar información agregada. No se trata de un conjunto de datos separado, sino de una consulta construida a partir de las operaciones registradas por el sistema.

V. RESULTADOS

Las pruebas finales permitieron comprobar que los módulos principales funcionan de manera integrada. El CRUD de medicamentos permite crear, consultar, editar y eliminar registros; el inventario refleja las existencias y marca los productos con stock bajo; y la gestión de proveedores mantiene la información necesaria para relacionar los medicamentos con sus distribuidores.

El control de acceso también produjo el comportamiento esperado. Con una cuenta de administrador se muestran las opciones administrativas y es posible gestionar usuarios y roles. En cambio, al ingresar como empleado o cliente, las funciones disponibles cambian y los intentos de abrir rutas no autorizadas muestran una pantalla de acceso denegado. Esto confirma que la separación de permisos no depende únicamente del menú visible.

En el flujo de compra se verificó la relación entre tienda, carrito, facturación e inventario. Una compra realizada desde una cuenta cliente generó una factura con su respectivo número, datos del comprador, detalle, impuesto y total. Después de la operación, el stock del medicamento utilizado disminuyó y la factura quedó disponible en el historial correspondiente.

Los reportes también reflejaron las ventas registradas. La consulta diaria presentó el número de facturas y el total vendido, mientras que el reporte por rango de fechas ordenó los medicamentos según la cantidad vendida. Estas pruebas muestran que la información generada por el proceso transaccional puede reutilizarse para consultas administrativas.

Finalmente, el despliegue permitió ejecutar el proyecto en un entorno diferente al equipo de desarrollo. La aplicación publicada se conecta a la base de datos remota y conserva los registros creados durante las pruebas. Con esto se obtuvo una versión demostrable mediante una URL de producción, además del repositorio con el historial del código y las evidencias.

A. Evidencias de funcionamiento

Las evidencias tomadas durante la revisión final documentan los estados más importantes del sistema: generación de facturas, disminución del stock, historial de ventas, restricciones de acceso, registro de cuentas, tienda, carrito, confirmación de compra, dashboard administrativo, gestión de usuarios, CRUD de medicamentos, proveedores y reportes. En conjunto permiten reconstruir el recorrido de una operación y comprobar la diferencia entre los perfiles.

Un ejemplo claro es la compra realizada desde una cuenta cliente. Primero se observa el medicamento disponible en la tienda, luego el producto dentro del carrito y posteriormente el formulario de confirmación. Después de completar la operación aparece la factura generada y el historial del cliente muestra únicamente su compra. Al revisar el inventario desde un perfil autorizado, la cantidad disponible refleja la unidad vendida.

B. Correspondencia con los requerimientos

Los resultados obtenidos cubren los grupos principales definidos por las historias de usuario: gestión de clientes, catálogo de medicamentos, proveedores, inventario, facturación, usuarios y roles, reportes e historial. Algunas historias se materializan mediante un módulo específico y otras se cumplen como parte de un flujo mayor. La

actualización automática del inventario, por ejemplo, se ejecuta como consecuencia de confirmar una venta.

La separación de roles también responde a los requerimientos de seguridad. El administrador mantiene el control sobre la gestión general; el empleado puede desarrollar las tareas operativas que le corresponden; y el cliente dispone de un recorrido de compra independiente. Las pruebas de acceso directo a rutas administrativas confirmaron que un enlace escrito manualmente en el navegador no permite omitir estas restricciones.

C. Repositorio y versión de producción

El repositorio conserva el código fuente y el historial de cambios del proyecto. Además de servir como medio de entrega, permite mostrar durante la defensa cómo evolucionó la aplicación y cuál es el commit correspondiente a la versión presentada. La carpeta de evidencias forma parte de ese mismo repositorio, por lo que las capturas quedan asociadas a la versión final del sistema.

La URL de producción permite realizar la demostración sin depender del servidor local. Al trabajar con la base de datos remota, las operaciones realizadas durante la defensa pueden comprobarse desde diferentes módulos de la misma aplicación. Esta característica resulta útil para evidenciar que la persistencia, la seguridad y el flujo de facturación se mantienen en el entorno desplegado.

D. Pruebas por perfil de usuario

Para el perfil ADMIN se comprobó el acceso al dashboard y a los módulos de usuarios, medicamentos, clientes, proveedores, inventario, facturas y reportes. También se verificó la modificación de roles y las operaciones CRUD disponibles en los módulos de mantenimiento. Este perfil representa la mayor cantidad de privilegios dentro del sistema.

Con el perfil EMPLEADO se revisó el acceso a las tareas operativas y, de manera específica, la imposibilidad de utilizar funciones reservadas al administrador. Se realizaron ventas y se consultaron facturas e inventario. Al escribir manualmente rutas administrativas en el navegador, el sistema respondió con la vista de acceso denegado.

Con el perfil CLIENTE se probó el registro, inicio de sesión, navegación por la tienda, adición de productos al carrito, confirmación de compra y consulta del historial propio. También se intentó ingresar a usuarios y reportes mediante la URL. La aplicación mantuvo las restricciones y no mostró información administrativa.

E. Prueba integrada de una venta

La prueba integrada más representativa comenzó con un medicamento disponible en la tienda. Se agregó una unidad al carrito y se revisó el resumen de cantidades y total. Después se continuó al formulario de compra, donde se ingresaron los datos del comprador y se seleccionó el método de pago. Al confirmar, la aplicación generó la factura y vació el flujo pendiente del carrito.

Posteriormente se abrió el historial de compras para comprobar que la nueva factura estuviera registrada. Desde un perfil autorizado se revisó el inventario y se confirmó la disminución de una unidad. Finalmente, el administrador consultó los reportes y la venta apareció dentro del total del día. Esta secuencia permitió comprobar en una sola prueba la

interacción entre tienda, carrito, venta, detalle, factura, inventario e informes.

F. Evidencias seleccionadas para el informe

Las treinta capturas conservadas en el repositorio sirven como respaldo completo de las pruebas, pero para el cuerpo del artículo conviene utilizar únicamente las que aportan información distinta. Las evidencias más representativas son el dashboard administrativo, la gestión de usuarios y roles, el CRUD de medicamentos, la vista de acceso denegado, el carrito, la factura generada y el reporte de ventas. El resto permanece disponible en la carpeta de evidencias para revisión.

Esta selección evita convertir el artículo en una colección de pantallas y mantiene el enfoque técnico. Durante la defensa, las demás capturas pueden utilizarse como apoyo si se solicita demostrar una función específica o explicar el comportamiento de un perfil.

G. Limitaciones y trabajo pendiente

La versión desarrollada cubre los flujos principales planteados para el proyecto, pero existen posibilidades de ampliación. Los reportes podrían incorporar más criterios de análisis y exportación; el inventario podría conservar un historial más detallado de entradas y salidas; y la anulación de facturas podría complementarse con un registro de auditoría más amplio.

También sería posible integrar mecanismos externos de pago o notificación, aunque para el alcance académico se mantuvieron métodos simulados de pago y retiro en tienda. Estas ampliaciones no son necesarias para demostrar el funcionamiento actual, pero representan una continuación natural de la arquitectura implementada.

H. Trazabilidad entre requerimientos y módulos

La implementación mantiene una relación directa entre los requerimientos funcionales y las pantallas disponibles. Las historias de clientes se reflejan en el módulo de consulta y registro; las historias de medicamentos se cubren con el mantenimiento del catálogo y la consulta de inventario; las relacionadas con proveedores se atienden desde su CRUD; y las de facturación se distribuyen entre carrito, confirmación de compra, factura e historial. Esta correspondencia facilita explicar durante la defensa por qué existe cada módulo y qué necesidad resuelve.

Las historias de seguridad se relacionan con el registro de usuarios, inicio de sesión, asignación de roles y protección de rutas. Los reportes responden a la necesidad de consultar ventas por fecha y productos con mayor salida. De esta forma, la solución final no se diseñó como un conjunto de páginas independientes, sino como una implementación de los requerimientos definidos al inicio del proyecto.

I. Consistencia del flujo de información

Uno de los puntos revisados con mayor cuidado fue evitar duplicar información sin necesidad entre módulos. El medicamento utilizado en la tienda corresponde al mismo registro que se consulta desde inventario y que se utiliza al crear el detalle de una venta. De manera similar, las facturas del historial provienen de las ventas registradas y los reportes se construyen a partir de esas operaciones.

Esta consistencia es importante porque permite verificar una acción desde diferentes perspectivas. Si se vende una unidad, el cambio puede observarse en la factura, en el historial, en el inventario y en el reporte. Si cada pantalla utilizara información independiente, sería posible obtener resultados contradictorios. La integración lograda reduce ese problema y constituye uno de los resultados principales del proyecto.

J. Separación entre interfaz y reglas de negocio

La interfaz se utilizó para capturar y presentar información, mientras que las decisiones importantes permanecen en el servidor. Por ejemplo, ocultar una opción del menú para un cliente mejora la experiencia de uso, pero no se consideró suficiente como medida de seguridad. La autorización de la ruta se mantiene en la configuración y en los componentes correspondientes del backend.

El mismo criterio se aplicó al proceso de compra. El navegador presenta cantidades y totales para que el usuario comprenda la operación, pero la persistencia de la venta y la modificación del inventario se realizan desde la lógica del servidor. Esto permite mantener las reglas esenciales fuera de elementos que pueden modificarse desde el cliente.

K. Preparación para la defensa técnica

La estructura final del proyecto permite demostrar el sistema siguiendo un recorrido corto y comprensible. Se puede iniciar con el dashboard del administrador para explicar los módulos y después mostrar un CRUD. A continuación, se cambia a un perfil con menos permisos para comprobar la seguridad y finalmente se realiza una compra desde la tienda hasta obtener la factura. El cierre de la demostración puede hacerse revisando el nuevo stock y el reporte del día.

Este recorrido permite relacionar la interfaz con el código. Para cada paso puede identificarse la vista utilizada, el controlador que recibe la solicitud, el servicio que ejecuta la operación y el repositorio que persiste la información. La demostración de la API REST en Postman complementa este recorrido al mostrar una interacción directa con el backend sin depender de la interfaz web.

L. Escenarios de validación final

Además de revisar cada módulo de forma individual, se definieron escenarios que combinan varias funciones. El primer escenario corresponde a la administración del catálogo: un administrador inicia sesión, consulta los medicamentos, registra o modifica un producto y revisa el resultado en inventario. El segundo escenario corresponde a seguridad: se ingresa con un perfil empleado o cliente y se intenta acceder a una ruta administrativa. El tercer escenario corresponde a la venta completa y conecta tienda, carrito, checkout, factura, inventario e historial.

Estos escenarios fueron útiles porque reproducen acciones cercanas al uso real del sistema. También permiten detectar errores que no siempre aparecen cuando se prueba una pantalla por separado. Una factura puede mostrarse correctamente, por ejemplo, pero la prueba no está completa hasta confirmar que el stock cambió y que el registro permanece disponible después de navegar a otro módulo.

M. Datos mostrados en producción

Las pruebas realizadas en el entorno desplegado utilizaron registros de medicamentos, proveedores, usuarios y ventas que permiten observar el comportamiento del sistema. El dashboard muestra quince medicamentos en catálogo, alertas de stock bajo y proveedores activos. Las pantallas de inventario y medicamentos presentan las cantidades actuales, mientras que las facturas registradas permiten comprobar los totales y el historial.

Los datos de demostración también permiten verificar el reporte de ventas. Las facturas generadas durante las pruebas aparecen en la consulta correspondiente a la fecha utilizada y los medicamentos vendidos se reflejan en el ranking por cantidad. Esto ofrece una evidencia visible de que los reportes utilizan información persistida y no valores estáticos colocados únicamente para la interfaz.

N. Mantenibilidad de la solución

La división en controladores, servicios y repositorios deja una base que puede ampliarse sin rehacer por completo la aplicación. Si se incorpora un nuevo reporte, por ejemplo, puede agregarse la consulta necesaria y su presentación sin modificar el funcionamiento del carrito. De igual manera, una mejora en la interfaz de medicamentos no requiere trasladar la lógica de persistencia a la vista.

El uso de entidades y relaciones también facilita mantener la coherencia del modelo. Las ventas conservan sus detalles y los medicamentos mantienen su relación con proveedores. Esta estructura es más apropiada que almacenar toda la información de una operación en un único registro, ya que una factura puede contener varias líneas de productos y cada medicamento puede participar en múltiples ventas a lo largo del tiempo.

O. Seguridad como parte del funcionamiento

La seguridad se trató como una característica funcional del sistema y no como un agregado final. El usuario autenticado determina qué opciones aparecen, pero la protección real se mantiene en las rutas. La pantalla de acceso denegado utilizada en las pruebas evidencia que el servidor reconoce una solicitud realizada por un usuario válido que, aun así, no posee el permiso requerido.

Esta diferencia es importante en la defensa técnica: autenticación responde a quién es el usuario, mientras que autorización determina qué puede hacer. Farmacia Vida utiliza ambas. Primero se valida la cuenta mediante el inicio de sesión y después se aplican las reglas correspondientes a ADMIN, EMPLEADO o CLIENTE para cada área de la aplicación.

VI. CONCLUSIONES

El proyecto permitió integrar en una sola aplicación procesos que normalmente se relacionan entre sí dentro de una farmacia: catálogo, clientes, proveedores, inventario, ventas y facturación. La principal mejora frente a manejar estos datos por separado es que una venta registrada tiene efecto directo sobre las existencias y permanece disponible para consultas posteriores.

La organización por capas facilitó mantener separadas la presentación, la atención de solicitudes, la lógica de negocio y la persistencia. De igual manera, Spring Security permitió aplicar permisos distintos para administrador, empleado y

cliente, incluyendo la protección de rutas cuando un usuario intenta ingresar directamente mediante la URL.

Las pruebas realizadas muestran que los flujos principales cumplen su propósito: los CRUD administrativos funcionan, el cliente puede completar una compra, la factura conserva su detalle, el inventario se actualiza y los reportes utilizan las ventas registradas. El despliegue en producción permitió comprobar además que el sistema puede operar con una base de datos remota y no depende únicamente de la configuración local.

Como trabajo futuro se podrían ampliar los reportes, incorporar mecanismos adicionales de auditoría y mejorar la gestión de anulaciones y movimientos de inventario. Estas mejoras pueden construirse sobre la estructura actual sin cambiar el flujo principal ya implementado.

REFERENCIAS

- [1] Spring, “Spring Boot Reference Documentation,” [Online]. Available: <https://docs.spring.io/spring-boot/>
- [2] Spring, “Spring Security Reference,” [Online]. Available: <https://docs.spring.io/spring-security/reference/>
- [3] Spring, “Spring Data JPA,” [Online]. Available: <https://spring.io/projects/spring-data-jpa>
- [4] Thymeleaf, “Documentation,” [Online]. Available: <https://www.thymeleaf.org/documentation.html>
- [5] Oracle, “MySQL Documentation,” [Online]. Available: <https://dev.mysql.com/doc/>
- [6] Bootstrap, “Documentation,” [Online]. Available: <https://getbootstrap.com/docs/>
- [7] Render, “Documentation,” [Online]. Available: <https://render.com/docs>
- [8] Aiven, “Documentation,” [Online]. Available: <https://aiven.io/docs>